

ual:

Coding Three

Creative Robotics

Week 2: 9 October 2025

Today's Agenda

■ Lecture:

- Assessments
- Recap: Images as Numpy arrays
- Working with webcams in OpenCV
- Core Computer Vision techniques (Erosion, Dilation, Blob Detection)
- Background Subtraction and Contour Detection
- Limitations of Traditional Computer Vision —> Introduction to Machine Learning ✨

■ Lab time



Assessment



What You Have to Produce

- You can find this unit's Assessment Brief on Moodle
- **Element 1: In-class assignments:**
 - You will be set weekly assignments to test your understanding of the week's topics (50%).
- **Element 2: Mini project:**
 - A mini project as specified in class (50%)
 - More detailed brief will be coming soon
 - Today's Mini Project brief is also uploaded to Moodle

Learning Outcomes

- **LO1** Understand how machine learning works in practice (**Knowledge**)
- **LO2** Demonstrate practical skills in computer programming for use in Computer Vision contexts (**Process**)
- **LO3** Use existing frameworks and examples to create computational systems that make use of Computer Vision approaches relevant to robotics (**Realisation**)

What You Have to Produce

- Your **Mini Project** will be assessed based on these three Learning Outcomes.
- There are three anonymized sample student projects from the previous course (B, A, and A+) uploaded to Moodle. However, the file labels do not correspond to which project received which grade
 - **Let's have a look and discuss which is which**
 - **Please note:** Last year's submission guidelines are a little different than this year's. So use these samples as an inspiration, but double-check your work before submissions to make sure you are turning in everything that is asked. Here is the **additional brief on Element 2**.
- You can find these sample projects on Moodle under Unit Guidance.

Mini Project

- You will work on a Mini Project of your own throughout the Unit
- A Mini Project should be:
 - A small creative project made with computational systems and primarily based on Python code
 - An application of *at least* one computer vision model that is based on machine learning in creative practice
 - Made relevant to robotics (this does **not** necessarily need to include a component for robotics or physical computing in the project)

Mini Project

- There will be sessions during the Unit that support the progress of your Mini Project, the activities of the sessions include but are not limited to:
 - In-class planning and designing
 - In-class making
 - Feedback from peers and tutor(s)
 - Surgery: Tweaking and improving code or the project's operation with a tutor

Mini Project Submission

- For Assessment, at the end of the Unit, you will submit:
 - Digitally presented outcomes of your Mini Project, its documentation of the process of development, in a digitally edited document titled *Element 2 Submission*.
 - The *Element 2 Submission* document should contain images, texts and optionally links, videos, audio and any other relevant media files to explain your project
 - Within your *Element 2 Submission* document, it should also include your own GitHub Repository link that contains **all the coding files related to this project**

Mini Project Submission

- In the *Element 2 Submission* document, please provide
 - **Documentation of the progress and outcomes** of the Mini project
 - **A link containing your GitHub Repository with all the coding files related to this project**
 - You don't need to include the ML model you have used for your project, *however*, you should reference to them in your write up so that we can see what ML model you chose for your work and your rationale on choosing this particular model
 - **Text in two parts:**
 - Project description (400 words maximum)
 - Reflective evaluation of the project development (500 words maximum)

Mini Project Submission

- Documentation of the outcomes of the Mini project should provide:
 - A clear explanation of how your Mini Project operates and functions
 - The progression of your Mini Project from conception to finish
 - You can use images and videos with annotations explaining what they do/represent
 - It should also present the trajectory of how you developed your project from initial ideas or prototypes to their variations to the final version
 - Keep your annotations brief: 50-80 words maximum per annotation, unless the use of more words is reasonably appropriate. For instance, to explain a complex mechanism.

Project Description

■ Should describe:

- What you made
- Which technical approaches are used
- How machine learning is used in the project
- Why you chose to employ particular methods (if relevant)
- How the project can be seen as relevant **to the context of robotics** i.e. either clarified through your explanation of how the chosen computer vision model can operate with a particular robotic system and its functionalities, or by demonstratively integrating robotic components with the computer vision used (the latter is an optional approach)

Reflective Evaluation

- Should describe **how you researched and experimented with different tools and methods** related to computer vision and why you chose the final methods from different options
- To work on this, you might start with questions such as:
 - What went well and what did not go well through your experimentation or tests
 - What have you managed to improve and how?
 - How effectively did the chosen machine learning functions work in your project?
 - How relevant are your project and computer vision approaches to the context of (creative) robotics?



OpenCV Fundamentals



What is OpenCV? (Quick Recap)

- OpenCV stands for Open Source Computer Vision Library
- It is an open-source computer vision and machine learning software library
- OpenCV was built to provide a common infrastructure for computer vision applications and to accelerate the use of machine perception in the commercial products.

- Official site: <https://opencv.org/>
- Documentation: <https://docs.opencv.org/>

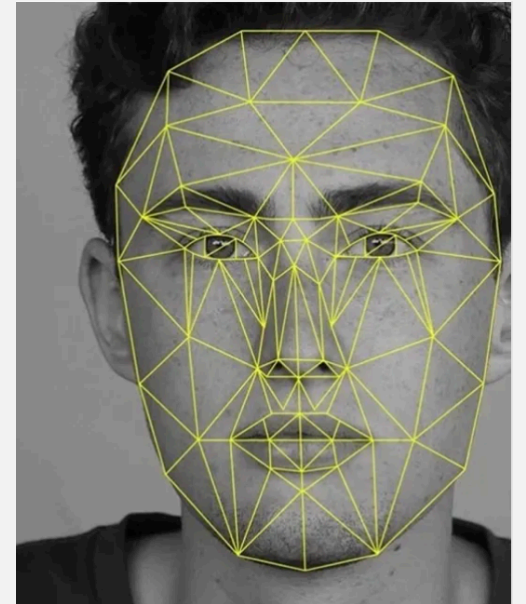
OpenCV Key Features

- Over 2500 optimized algorithms
- Interfaces for Python, C++, Java, MATLAB
- Cross-platform (Windows, Linux, macOS, Android, iOS)
- Real-time computer vision capabilities
- Extensive documentation and community support

Why OpenCV Matters for Robotics

In creative robotics, OpenCV serves as the "eyes" of your robotic systems:

- **Perception:** Detecting objects, tracking motion, recognizing patterns
- **Navigation:** Identifying obstacles, mapping environments
- **Interaction:** Responding to human gestures, facial expressions
- **Real-time processing:** Critical for responsive robotic behavior



What is Computer Vision?

Computer Vision = Teaching computers to "see" and understand images

Think about what happens when you look at a photo:

- You recognize faces, objects, colors
- You can tell what's moving vs. what's still
- You understand depth and distance

Computer vision helps machines do the same things

Real World Examples of Computer Vision

- Face unlock on your phone
- Robots navigating spaces
- Self-driving cars detecting pedestrians
- Instagram filters and effects
- Medical image analysis

NumPy Arrays

What We Will Learn in This Section

- Why NumPy matters for computer vision
- Images as arrays
- Basic array operations
- We will be using the `j_array_test.py` lab together

Quick Note on Installation: conda vs. venv

- Last week, we set up a conda environment `coding3` for your coursework
- You may also see instructions that use Python's `venv` + `pip`. This works fine for many pure Python projects.
- venv = "virtual environment": Python's built-in tool for creating isolated environments so each project has its own Python interpreter and `pip`-installed packages.
- **conda**: Environment manager that isolates Python (or other) environments and installs packages, which include non-Python binary dependencies and compiled libraries

Quick Note on Installation: conda vs. venv

- If you already prefer venv and can install everything successfully, feel free to use it
- However, if you run into build errors (especially with packages that need compiled binaries), please switch to the provided conda environment
- If you don't want to use either of these to work in your local machine, you can also use online notebooks like Binder/Colab, where you can upload your Jupyter notebooks online
 - Using online tools can be slower, limited data and resources (you may have to pay after a certain usage amount), privacy issue

Updating Conda Environment to Python 3.10

- Python 3.9 recently reached end-of-life, and VS Code/Jupyter now shows warnings when using it.
- This is part of the nature of programming. Software and dependencies get retired and updated all the time
- To learn how to debug is a critical skill to have
- This has happened to me prior to hand-in during my MSc at CCI
- If this happens prior to your final hand-in, just note this in your documentation about why the project did not accomplish what you set out to do originally

NumPy Arrays: The Foundation

What is an Array?

An **array** is like a grid of numbers:

```
# 1D array (a row of numbers)
```

```
[1, 2, 3, 4, 5]
```

```
# 2D array (a table)
```

```
[[1, 2, 3],
```

```
 [4, 5, 6],
```

```
 [7, 8, 9]]
```

```
# 3D array (a stack of tables)
```

```
Think: a deck of cards, each card is a 2D table
```

NumPy Arrays: The Foundation

What is an NumPy?

NumPy is a Python library the provides fast, efficient arrays and numerical operations (vectors, matrices, math functions) that are core to scientific computing in Python

- Fast array operations
- No Python loops needed
- **OpenCV users NumPy arrays for EVERYTHING**

Images as NumPy Arrays: The Foundation

Python Lists vs. NumPy Arrays

Python Lists:

```
my_list = [1, 2, 3, 4, 5]
# To add 10 to everything: SLOW
for i in range(len(my_list)):
    my_list[i] = my_list[i] + 10
```

NumPy Arrays:

```
import numpy as np
my_array = np.array([1, 2, 3, 4, 5])
# To add 10 to everything: FAST
my_array = my_array + 10 # Done!
```

Key difference: NumPy does math on ALL elements at once (called “vectorization”), which increases speed of processing for real-time processing with simple syntax

Images as NumPy Arrays: The Foundation

```
1
2  import cv2 as cv
3  import numpy as np
4
5  # Load an image
6  image = cv.imread('image/tamagotchi1.jpeg', cv.IMREAD_COLOR)
7
8  # What is it?
9  print(type(image))          # <class 'numpy.ndarray'>
10 print(image.shape)          # (height, width, 3), in this case, it's (480, 623, 3)
11 print(image.dtype)          # uint8 (0-255 for each channel)
```

You can practice this yourself in today's lab notebook `j_array_test.py`

Understanding the Image Shape

```
10 print(image.shape)          # (height, width, 3), in this case, it's (480, 623, 3)
```

```
13 # Example: (480, 623, 3) means:  
14 # - 480 pixels tall  
15 # - 623 pixels wide  
16 # - 3 color channels (Blue, Green, Red)  
17  
18 # Total elements  
19 print(image.size)           # 897,120 (480 × 623 × 3)
```

 **The format is (height, width, channels), not (width, height)**

Images as NumPy Arrays: The Foundation

Critical Concept: In OpenCV, each image is NumPy N-dimensional array (`ndarray`)

You can use ALL NumPy operations on images

For example:

- **Fast indexing:** `image[100, 100]` uses NumPy's optimized array operations
- **Efficient slicing:** `image[50:200, 50:200]` creates views, not copies
- **Vectorized operations:** Process entire images without Python loops

To run today's notebooks

- **.py** extension means that they are Python files
- To run them, you can open Terminal/Command Prompt:
- First, activate your conda in your command line
 - **conda activate coding3**
- Then, **cd** to the folder that your files are downloaded to
 - For example: **cd path/to/week2**
- You can find your file folder path by right clicking the folder and copy the **Relative Path**
- Then, you can type in this in your command line: **python j_array_test.py**

To run today's notebooks

- You can also do the same from Visual Studio Code
- Go to the Explorer and open the folder that contain today's class files
- Go to View on the top Menu bar —> Terminal (or Ctrl+`)
- Activate the conda environment:
 - **conda activate coding3**
- Then, run the script: **python j_array_test.py**

Loading Images

Before We Get Started

- The goal today is not to overwhelm you with coding, but to help you understand what does each function do and how they relate to the bigger picture.
- By understanding each component in the foundation areas, this will help you with more advanced concepts later
- We are not expecting you to become coding expert through this course, but know and understand enough to use it as a tool to create and express your work

What We Will Learn in This Section

- Loading and displaying images
- OpenCV's BGR color format
 - *Not* RGB!
 - OpenCV stores image arrays in BGR order because many early camera and video APIs and file formats (and Windows bitmap conventions) used BGR, so OpenCV adopted that ordering for compatibility and performance.
- Saving images
- Basic image properties
- We will run **`a_image_show.py`** together

Quick Installation Check

■ Let's double check your installation before we start today

```
bash

# Activate environment
conda activate coding3

# Test import
python
>>> import cv2 as cv
>>> print(cv.__version__)
4.9.0 # (your version)
>>> exit()
```

Lab: a_image_show.py: Loading

■ Let's look at the code together. Here is Line 18 of your file:

```
img_loaded = cv.imread(file_path, cv.IMREAD_COLOR)
```

cv.imread() = Read image from file; second parameter options:

cv.IMREAD_COLOR: Load as color (default)

cv.IMREAD_GRAYSCALE: Load as grayscale

cv.IMREAD_UNCHANGED: Load with alpha channel (If the image has an alpha channel, which controls the transparency of the image)

Lab: a_image_show.py: Error Checking

■ Let's look at the code together: Line 20:

```
if img_loaded is None:  
    exit("Could not read the image.")
```

- If the file doesn't exist, **imread()** returns **None**
- If you don't check —> crash later with confusing error
- Always check if your image loads successfully

Lab: a_image_show.py: Display

■ Let's look at the code together: Line 24:

```
# Display the image  
cv.imshow("window", img_loaded)
```

■ **cv.imshow()** = Display image in window

■ First parameter: window name (string)

■ Second parameter: image array

Lab: a_image_show.py: Display

■ Let's look at the code together: Line 27:

```
# Wait for key press  
close_with_a_key = cv.waitKey(0)
```

■ **cv.waitKey(0)** = Wait for key press

■ 0 = wait forever

■ 1000 = wait 1000 milliseconds (1 second)

■ If there is no argument in () then the window closes immediately

Lab: a_image_show.py

- To run the code, go to your command line (Terminal/ Command Prompt)
- **cd** to the folder containing the file
- Type in:
 - **python a_image_show.py**

Lab: a_image_show.py

■ What happens:

1. Image loads
2. Window appears
3. You can press any key to close
4. You can also press 's' to save as PNG

⚠ Important: BGR vs RGB

■ OpenCV uses BGR (Blue, Green, Red), NOT RGB!

```
python
```

```
# Create a red pixel in OpenCV
```

```
red_pixel = [0, 0, 255] # BGR: Blue=0, Green=0, Red=255
```

```
# NOT [255, 0, 0] - that would be BLUE!
```

! Important: BGR vs RGB

■ When this matters:

- Using OpenCV with other libraries like matplotlib, PIL, web browsers, which expect RGB
- If you plot an OpenCV image without converting, colors will look wrong (blues and reds swapped)
- Color detection tasks
- `cv.cvtColor()` = Convert color space
- This reorders the channels only, not resizing

Converting Color Spaces

```
python
```

```
# BGR → RGB
```

```
img_rgb = cv.cvtColor(img, cv.COLOR_BGR2RGB)
```

```
# BGR → Grayscale
```

```
img_gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
```

```
# RGB → BGR
```

```
img_bgr = cv.cvtColor(img_rgb, cv.COLOR_RGB2BGR)
```

Webcam Basics

What We Will Learn in This Section

- Capturing video from webcam
- The video loop pattern
- Flipping and displaying frames
- Controlling frame rate
- We will run `c_webcam_video.py` together

The Video Loop Pattern

- Initialize camera
- Loop:
 - Capture a frame
 - Process the frame (e.g., convert color, detect/transform)
 - Display the frame
 - Check for exit key or condition
- Release camera

The Video Loop Pattern

- In this process, the processing time determines frame rates
- If processing takes longer than the capture interval, you get a lag or dropped frames
- This is why traditional/classical CV can be slow when it uses computationally expensive steps
- Machine learning (especially optimized deep models and hardware acceleration) is often applied for:
- Improve accuracy for tasks (detection, segmentation, tracking)

Where ML Comes In

- Machine learning (especially optimized deep models and hardware acceleration) is often applied for:
 - Improve accuracy for tasks (detection, segmentation, tracking)
 - Replace complex pipelines with a single learned model
 - Allows approximate but faster solutions

Lab: c_webcam_video.py: Initialization

■ Let's look at the code together. Here is Line 4 of your file:

```
# Initialize camera (0 = default camera)  
capture = cv.VideoCapture(0)
```

cv.VideoCapture() = Connect to camera

0: Load as color (default)

1: Load as grayscale

Can also be a video file path: “**video.mp4**”

Lab: c_webcam_video.py: Initialization

■ Let's look at the code together. Here is Line 6 of your file:

```
if not capture.isOpened():  
    print("Cannot open camera")  
    exit()
```

Below, we have a check to see if the camera opened successfully.

It's always a good practice to check!

Lab: c_webcam_video.py: Window Setup

■ Let's look at the code together. Here is Line 11-12 of your file:

```
# Create resizable window  
cv.namedWindow('frame', cv.WINDOW_NORMAL)  
cv.resizeWindow('frame', 800, 600)
```

cv.namedWindow() = Create window with name

cv.WINDOW_NORMAL() = resizable window

cv.WINDOW_AUTOSIZE() = fixed size (default)

cv.resizeWindow() = Set window size: width, height in pixels

You can use **cv.imshow()** directly without this

Lab: c_webcam_video.py: The Loop

■ Let's look at the code together. Here is Line 14-15 of your file:

```
while True:  
    ret, frame = capture.read()
```

capture.read() = Grab one frame

Returns two values:

■ **ret**(boolean): True if frame captured successfully

■ **frame**(array): The actual image

Lab: c_webcam_video.py: The Loop

■ Let's look at the code together. Here is Line 17-19 of your file:

```
if not ret:  
    print("Can't receive frame. Exiting ...")  
    break
```

Check **ret**: if camera disconnects, exit

Lab: c_webcam_video.py: Processing

■ Let's look at the code together. Here is Line 22 of your file:

```
# Flip horizontally (mirror effect)  
frame = cv.flip(frame, 1)
```

cv.flip() = Flip/mirror image

■ **1** = flip horizontally (left ↔ right)

■ **0** = flip vertically (up ↔ down)

■ **-1** = flip both directions

■ Why flip: Makes webcam act like a mirror (more intuitive for users)

Lab: c_webcam_video.py: Display & Exit

■ Let's look at the code together. Here is Line 24-28 of your file:

```
cv.imshow('frame', frame)

# ESC key (27) to exit
if cv.waitKey(30) == 27:
    break
```

cv.waitKey(30) = Wait 30 milliseconds; **always include waitKey()** in video/display loops or windows won't update/respond

■ Allows window to refresh

■ ~33 fps (1000ms / 30ms ≈ 33 frames per second)

■ Lower value = faster, higher value = slower

■ **27** = ASCII code for ESC key (meaning you press ESC key and the computer reports it as 27)

Lab: c_webcam_video.py: Cleanup

■ Let's look at the code together. Here is Line 30-31 of your file:

```
capture.release()  
cv.destroyAllWindows()
```

capture.release() = Disconnect from camera

■ Frees up camera and memory to run other programs

■ Develop the good practice where you always cleanup when you are done

cv.destroyAllWindows() = Close all OpenCV windows

Break (15 Minutes)

Morphological Operations

What We Will Learn in This Section

- What erosion and dilation do
- When to use each operation
- Kernel size and iterations
- Cleaning up images

- We will run **`d_morphology.py`** together

The Problem: Noisy Images

- Real-world images have issues:
 - Random noise (speckles, dots)
 - Gaps in detected shapes
 - Unwanted small objects
 - Rough edges

The Problem: Noisy Images

- Example scenarios:
 - Detecting a hand → background noise appears
 - Finding a ball → small holes in detection
 - Tracking motion → jittery, broken shapes
- This is where we use morphological operations to mitigate these

What are Morphological Operations?

- Morphological = shape-based processing
- Think of geological processes:
 - Erosion: Water wears away rock (shrinks shapes)
 - Deposition: Sediment builds up (grows shapes)
- In images:
 - Erosion: Shrinks bright regions, removes small bright spots
 - Dilation: Expands bright regions, fills small gaps
- Both use a **kernel** (small grid) that slides across the image

Comparison



https://docs.opencv.org/4.x/Morphology_1_Tutorial_Theory_Original_Image.png

https://docs.opencv.org/4.x/Morphology_1_Tutorial_Theory_Dilation.png

https://docs.opencv.org/4.x/Morphology_1_Tutorial_Theory_Erosion.png

Comparison



Original



Dilation

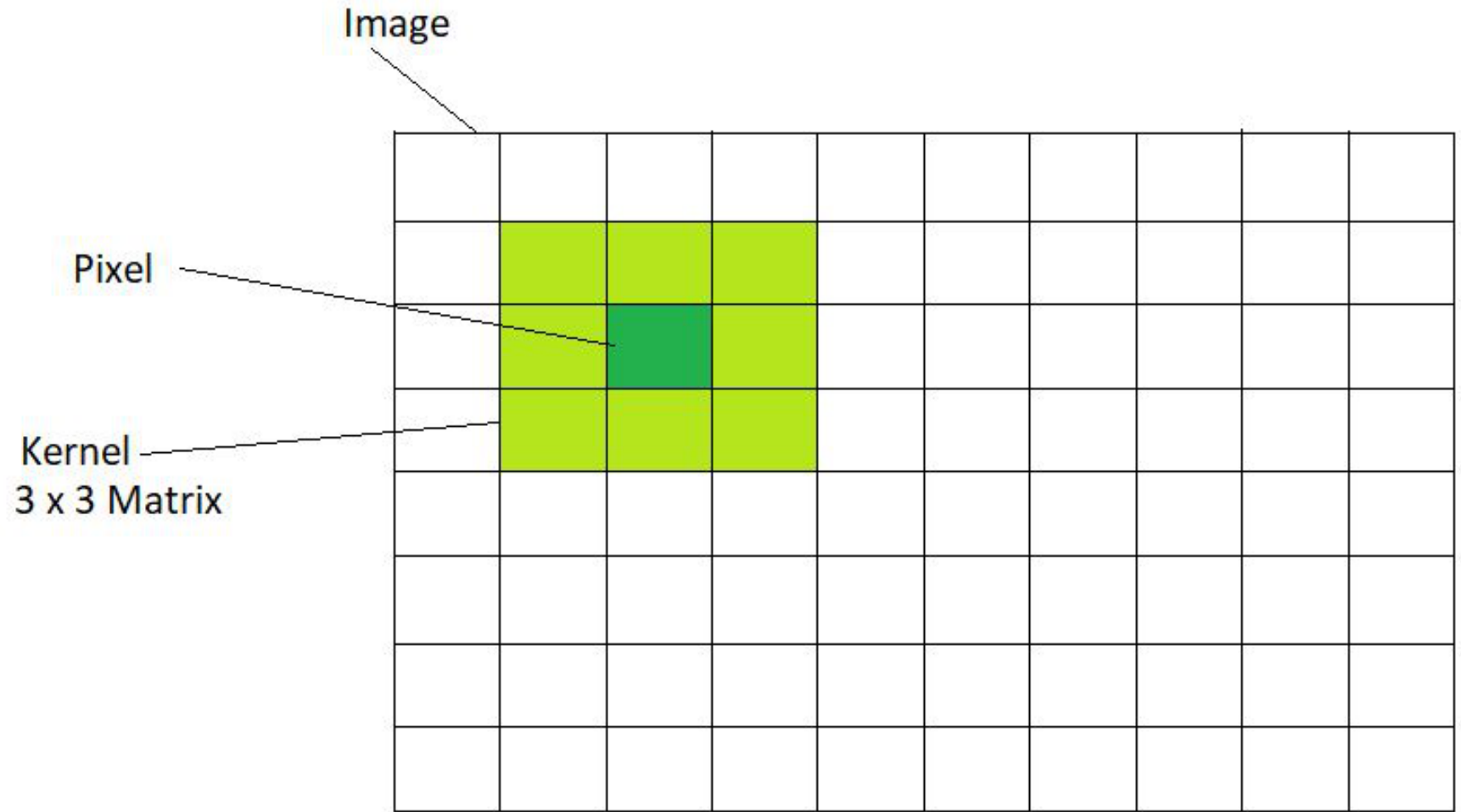


Erosion

The Kernel

- Both Erosion and Dilation use a **kernel** (small grid) that slides across the image
- Kernel is a small matrix that defines the operation
- The kernel slides across every pixel in the image and checks nearby pixels
- Different sizes = different strength effects
 - Small kernel = affects tiny details
 - Big kernel = affects larger shapes
- Kernel shape matters: a square, circle, or cross gives different effects

The Kernel



<https://www.geeksforgeeks.org/python/image-filtering-using-convolution-in-opencv/>

2D Convolution

The Kernel

- Think of it as the brush tool in Photoshop:
 - The kernel is like a tiny brush or stamp that you press over the image to change the center pixel based on nearby pixels
 - It checks a pixel and its neighbors as it slides across the image
 - Bigger brush (larger kernel) affects bigger areas
 - Different brush shapes (square, circular, cross) produce different effects

Erosion: How it Works

- If ANY pixel under the kernel is black (0), make center pixel black
- Effects:
 - Shrinks bright regions
 - Removes small bright spots
 - Separates touching objects
 - “Eats away” boundaries
- Think of it like sand washing away from a sandcastle

Erosion: What to Use

■ Use erosion to:

1. Remove small bright noise/speckles
2. Separate slightly connected objects
3. Shrink bright regions
4. Clean up thin bright connections

Erosion: When to Use

■ Use erosion to:

1. Remove small bright noise/speckles
2. Separate slightly connected objects
3. Shrink bright regions
4. Clean up thin bright connections

■ Example use case:

- Thresholded image has white noise specks
- Apply erosion → noise disappears
- Larger objects slightly shrink but remain

Dilation: How it Works

- If ANY pixel under the kernel is white (255), make center pixel white
- Effects:
 - Expands bright regions
 - Fills small gaps/holes
 - Connects nearby objects
 - "Grows" boundaries
- Think of it like adding clay to make something bigger

Dilation: What to Use

■ Use dilation to:

1. Fill small holes/gaps in shapes
2. Connect nearby objects
3. Expand bright regions
4. Make thin features more visible

Dilation: What to Use

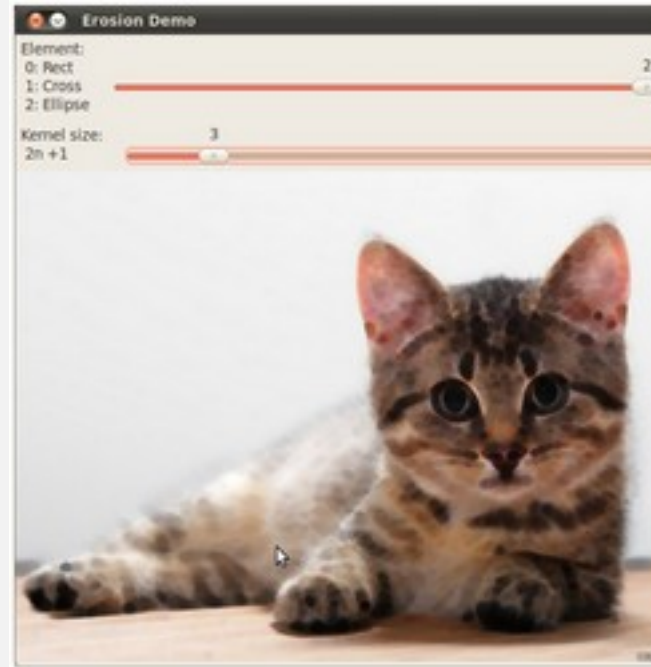
■ Use dilation to:

1. Fill small holes/gaps in shapes
2. Connect nearby objects
3. Expand bright regions
4. Make thin features more visible

■ Example use case:

- Detected hand has small gaps
- Apply dilation → gaps fill in
- Hand becomes solid shape

Why Is This Important for Creative Robotics?



https://docs.opencv.org/3.4/Morphology_1_Tutorial_Original_Image.jpg

https://docs.opencv.org/3.4/Morphology_1_Result.jpg

Why Is This Important for Creative Robotics?

- On the creative-robotics side, erosion and dilation are practical preprocessing tools to clean and shape the image/mask data so downstream tasks (tracking, contouring, object detection, motion control) work reliably.
- For example:
 - Remove sensor noise and small specks
 - Fill tiny holes in detected objects so area measures are stable
 - Separate touching objects so you can count or target them individually (erosion)
 - Smooth mask edges so robot path planning is more robust

Lab: d_morphology.py: Kernel

■ Let's look at the code together. Here is Line 12 of your file:

```
# Create 5x5 kernel  
kernel = np.ones((5,5), np.uint8)
```

np.ones(): creates a small array filled with ones

■ **(5,5)**: this is the shape of the kernel: 5 rows x 5 columns. The array is a 5x5 grid.

■ **np.uint8**: sets the data type to a 8-bit unsigned integers (value 0-2255), which OpenCV expects for kernels and binary images

Kernel Size Matters

■ Small kernel (3x3):

- Subtle changes
- Preserves detail
- Faster processing

■ Large kernel (11x11):

- Dramatic changes
- Removes more detail
- Slower processing

```
kernel_small = np.ones((3, 3), np.uint8)  
kernel_large = np.ones((11, 11), np.uint8)
```

■ Experiment to find the right size for your task!

Lab: d_morphology.py: Window Setup

■ Let's look at the code together. Here is Line 15 of your file:

```
erosion = cv.erode(img, kernel, iterations=1)
```

cv.erode() = Apply erosion

■ **img** = source image

■ **kernel** = structuring element

■ **iterations** = how many times to apply

Lab: d_morphology.py: Window Setup

■ Let's look at the code together. Here is Line 16 of your file:

```
dilation = cv.dilate(img, kernel, iterations=1)
```

cv.dilate() = Apply dilation

■ **img** = source image

■ **kernel** = structuring element

■ **iterations** = how many times to apply

■ *Same parameters as erode*

Iterations Matters

■ **iterations=1:**

- Mild effect
- Single pass through image

■ **iterations=5:**

- Strong effect
- Like applying the operation 5 times
- Much more dramatic change

■ Usually: 1-3 iterations is enough

Advanced: Opening & Closing

- **opening** = Erosion the Dilation
 - Removes small **bright** noise
 - Preserves object size

```
opening = cv.morphologyEx(img, cv.MORPH_OPEN, kernel)
```

- We will see these in **h_contour_bg_sub_webcam.py** later!

Advanced: Opening & Closing

- **closing** = Dilation then erosion

- Removes small **dark** noise

- Preserves object size

```
closing = cv.morphologyEx(img, cv.MORPH_CLOSE, kernel)
```

- We will see these in **h_contour_bg_sub_webcam.py** later!

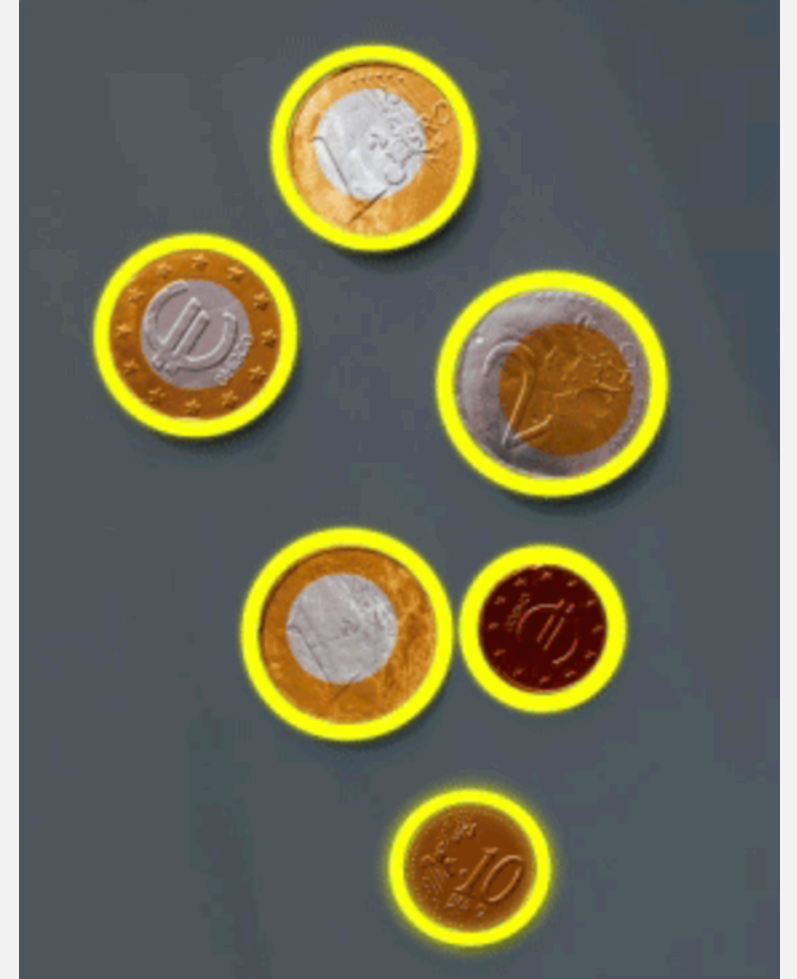
Blob Detection

What We Will Learn in This Section

- What blobs are
 - SimpleBlobDetector parameters
 - Detecting blobs in images
 - Real-time blob detection
 - Creative robotics applications
- We will run `e_blob_image.py` and `f_blob_webcam.py` together

What is A Blob?

- Blob = a region of connected pixels with similar properties
- Think of it as a “spot” or “object” in an image
- Examples:
 - A red ball against a wall
 - Your hand against a background
 - A dark spot on a light surface
 - A person’s face in a crowd
- Key ideas: Group of pixels that belong together



https://opencv.org/wp-content/uploads/2025/05/Blob_Detection_using_OpenCV.gif

Why Blob Detection?

- Use cases in robotics:
 - **Object tracking:** Follow a colored ball
 - **Gesture recognition:** Detect hand positions
 - **Navigation:** Find obstacles or markers
 - **Interaction:** Respond to objects in scene

SimpleBlobDetector

- OpenCV has a built-in blob detector with many parameters
- For example, you will see this starting at Line 46 in `e_blob_image.py`

```
# Setup SimpleBlobDetector parameters.
params = cv2.SimpleBlobDetector_Params()

# Change thresholds
params.minThreshold = min_threshold
params.maxThreshold = max_threshold

# Set minRepeatability to 1 to avoid the warning
params.minRepeatability = 1

# Filter by Area.
params.filterByArea = True
params.minArea = 500

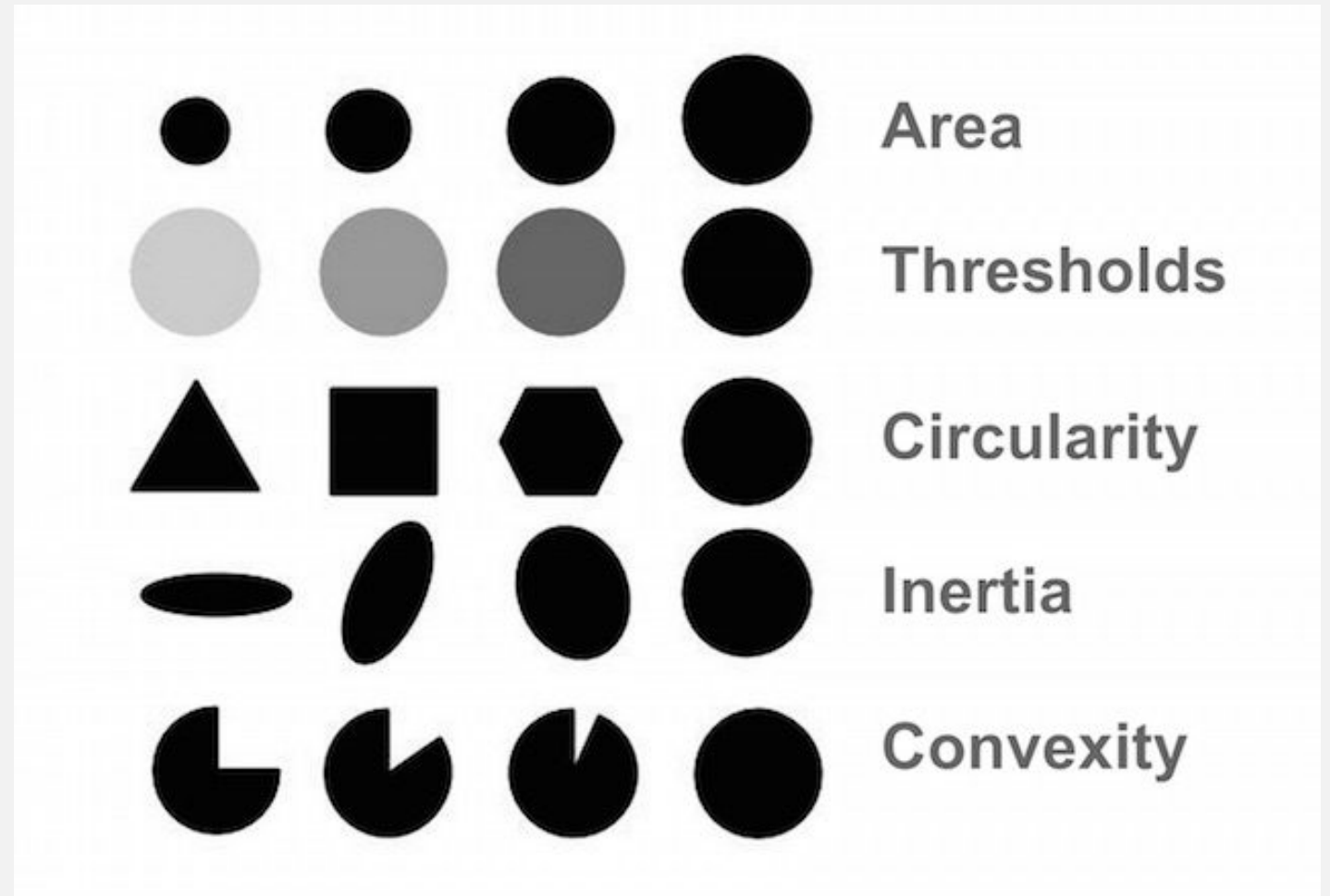
# Filter by Circularity
params.filterByCircularity = True
params.minCircularity = 0.5

# Filter by Convexity
params.filterByConvexity = True
params.minConvexity = 0.87

# Filter by Inertia
params.filterByInertia = True
params.minInertiaRatio = 0.01
```

Filtering Blobs

- The parameters for **SimpleBlobDetector** can be set to filter the type of blobs we want
- Parameters are set *once* before the loop (more efficient)



<https://opencv.org/wp-content/uploads/2025/05/BlobTest.jpg>

Key Parameters: Threshold

- **What it does:** Separates object from background
 - Works on grayscale images (0-255)
 - Lower threshold = detect lighter objects
 - Higher threshold = detect darker objects
- **Think of it as:** Brightness cutoff
- **Experiment:** Different objects need different thresholds

```
params.minThreshold = 10  
params.maxThreshold = 200
```

Key Parameters: Area

- **What it does:** Filters blobs by size
 - Rejects blobs that are too small (noise)
 - Rejects blobs that are too large (background)
- **Use case:** Only want medium-sized objects

```
# Filter by Area.  
params.filterByArea = True  
params.minArea = 500
```

Key Parameters: Circularity

■ **What it does:** Measures how “round” the blob is

■ **1.0** = perfect circle

■ **0.0** = line

■ **0.5** = somewhat circular

■ **Use case:** Detect balls, ignore hands

```
# Filter by Circularity
params.filterByCircularity = True
params.minCircularity = 0.5
```

Key Parameters: Convexity

- **What it does:** Measures how “smooth” vs “bumpy” the blob is
 - **1.0** = perfectly convex (smooth outline)
 - Lower values = more irregular/jagged
- **Use cases:** Smooth objects vs rough/complex shapes

```
# Filter by Convexity
params.filterByConvexity = True
params.minConvexity = 0.87
```

Key Parameters: Inertia

- **What it does:** Measures how elongated the blob is
 - **1.0** = circular
 - Lower values = more stretched/ elliptical
- **Use cases:** Distinguish between round and elongated objects

```
# Filter by Inertia  
params.filterByInertia = True  
params.minInertiaRatio = 0.01
```

Key Parameters: Color

- **What it does:** Look for dark or bright blobs
 - 0 = dark blobs on light background
 - 255 = light blobs on dark background
- **Use cases:** Black markers on white paper vs vice versa

```
# Filter by colour - 0 for dark blob 255 for a light blob
params.filterByColor = True
# params.blobColor = 0
# params.blobColor = 255
```

Let's run e_blob_image.py

■ What You'll See:

■ The window shows:

- Original grayscale image
- Detected blobs as RED circles
- Two trackbars to adjust thresholds

■ In terminal:

- Blob count
- Position (x, y) of each blob
- Size of each blob

What the Terminal Shows

- Terminal printouts show the numeric blob info we need
- But for robot control we must convert pixels → real coordinates, filter for stability, choose a target, and send the cleaned values to the robot controller rather than relying on printed lines
- You need something like this to control a robot:

```
for kp in keypoints:  
    x, y = kp.pt      # Position (center of blob)  
    size = kp.size    # Diameter of blob
```

Let's run `f_blob_webcam.py`

- What's different from the static image version:
 - Captures from webcam
 - Converts to grayscale
 - Applies erosion to clean noise
 - Detects blobs continuously
 - Only prints when blob info changes
- **Try it:** Hold up objects, move your hand, etc.

Why Grayscale Conversion?

■ Line 67:

■ Benefits:

- Simpler (1 channel instead of 3)
- Faster processing
- Blob detector works on grayscale
- Shape detection doesn't need color

```
# Convert to grayscale  
gray = cv.cvtColor(frame, cv.COLOR_BGR2GRAY)
```

Erosion Preprocessing:

■ Line 70-75:

■ Benefits:

- Removes small noise
- Cleans up detection
- More reliable blob detection

```
# https://docs.opencv.org/4.x/d9/d61/tutorial\_py\_morphology.html
# create a kernel to use erode/dilate w
kernel = np.ones((7,7),np.uint8)
# apply erosion or dilation
modified_grey = cv.erode(gray,kernel,iterations = 2)
# modified_grey = cv.dilate(gray,kernel,iterations = 1)
```

Background Subtraction

What We Will Learn in This Section

- What background subtraction is
 - MOG2 and KNN algorithms
 - Motion detection
 - Creating foreground masks
-
- We will run `g_background_sub_webcam.py` together

What is Background Subtraction?

- Detect moving objects by comparing them to the static background
- The Idea:
 - **Learn** what the background looks like (static parts)
 - **Compare** each new frame to the background
 - **Identify** anything different = foreground (moving)
 - **Create mask** showing only movement

Background Subtraction with OpenCV

- Can handle lighting changes, gradual changes and shadows, which the traditional approach could not
- Here are Lines 7 and 22 from the **g_background_sub_webcam.py** file

```
# Algorithm learns and updates background model  
backSub = cv.createBackgroundSubtractorMOG2()  
  
# Automatically adapts to changes  
fgMask = backSub.apply(frame)
```

Two Algorithms: MOG2 vs KNN

- Line 7 from the `g_background_sub_webcam.py` file
- MOG2 (Mixture of Gaussians)
- How it works:
 - Models each pixel as mixture of Gaussians
 - Good for gradual lighting changes
 - Default choice for most applications

```
# Create a background subtractor object: Background subtraction method – KNN or MOG2
backSub = cv.createBackgroundSubtractorMOG2()
```

Two Algorithms: MOG2 vs KNN

- Line 8 from the `g_background_sub_webcam.py` file

- KNN (K-Nearest Neighbors)

- How it works:

 - Uses K-nearest neighbors algorithm

 - Better at handling shadows

 - Slightly more accurate but slower

```
# backSub = cv.createBackgroundSubtractorKNN()
```

Understanding the Output: Foreground Mask

- **Foreground mask** = Binary image showing movement
- Line 22 from the `g_background_sub_webcam.py` file
- This mask becomes input for next steps (blob detection, contours)
- The background is static, and compares each new frame to that model to find pixels that changed —> those changes are likely moving objects (foreground)

```
fgMask = backSub.apply(frame)
```

Understanding the Output: Foreground Mask

- The foreground mask produced by background-subtraction algorithms is typically a single-channel grayscale image (not a color image).
- Each pixel is a brightness value that indicates how the algorithm classifies it:
 - 255 (white): foreground (moving / changed)
 - 0 (black): background (static)
 - Intermediate gray values (e.g., ~127): uncertain pixels or shadows, depending on the algorithm and parameters

How `backSub.apply()` Works

- Line 22 from the `g_background_sub_webcam.py` file
- What happens internally:
 - Algorithm looks at current frame
 - Compares to learned background model
 - Updates background model (adapts over time)
 - Returns foreground mask

```
fgMask = backSub.apply(frame)
```

How `backSub.apply()` Works

- Line 22 from the `g_background_sub_webcam.py` file
- `backSub` is a background-subtractor object
- `apply(frame)` updates the background model with the current frame and returns a foreground mask image.

```
fgMask = backSub.apply(frame)
```

How `backSub.apply()` Works

- Line 22 from the `g_background_sub_webcam.py` file
- Learning phase:
 - First ~100-200 frames: building background model
 - After that: stable detection
 - Continues updating: adapts to slow changes
- **Tip:** Stay still for first few seconds for best results!

```
fgMask = backSub.apply(frame)
```

Let's run `g_background_sub_webcam.py`

■ What you'll see:

- Original webcam feed
- Foreground mask (black/white)
- Your movement appears as white
- Static background becomes black

■ Try:

- Stay still → you disappear
- Wave your hand → white motion
- Walk in/out of frame

Random International: Rain Room

- Going back to this case study from last week, what CV techniques do you think was used?



Random International: Rain Room

- Going back to this case study from last week, what CV techniques do you think was used?
- Motion detection (background subtraction)
- Blob detection (finding people)
- Tracking (following movement)



Contours

What We Will Learn in This Section

- What contours are
- Finding contours in mask
- Drawing bounding boxes
- We will run `h_contour_bg_sub_webcam.py` together

What are Contours?

- A curve joining continuous points along a boundary
- **Think of it as:** Outline or edge of an object
- Examples:
 - Outline of your hand
 - Border of a detected blob
 - Edge of a moving person
- Why useful?
 - Get shape of object
 - Calculate position
 - Draw bounding boxes
 - Measure size/area

Finding Contours

```
contours, _ = cv.findContours(  
    fgMask,                # Input: binary mask  
    cv.RETR_EXTERNAL,      # Mode: only external contours  
    cv.CHAIN_APPROX_SIMPLE # Method: compress contour  
)
```

■ Line 33:

■ **cv.findContours()** returns:

■ **contours** = list of contours (each is array of points)

■ **_** = hierarchy (in many cases we don't need it, so ignore it by using _)

■ **RETR_EXTERNAL** = only outermost contours (ignore holes)

■ **CHAIN_APPROX_SIMPLE** = compress contour (save memory)

Drawing Bounding Boxes

■ Line 36 - 41:

■ **cv.contourArea()** = size of contour in pixels. In the code, it's > 1000 , meaning **cv.contourArea(contour)** computes the area in pixels enclosed by the contour and will ignore contours smaller than 1000 pixels (treats them like noise)

■ **cv.boundingRect()** = smallest rectangle that fits contour

■ Returns: x, y, width and height

■ **cv.rectangle()** = draw the box

■ Color: (159, 229, 202)

■ 2 is the line thickness in pixels

```
# Draw bounding boxes around detected objects
for contour in contours:
    if cv.contourArea(contour) > 1000: # Adjust this threshold as needed
        x, y, w, h = cv.boundingRect(contour)
        cv.rectangle(frame, (x, y), (x+w, y+h), (159, 229, 202), 2)

    # print(contour)
```

Let's run `h_contour_bg_sub_webcam.py`

■ What you'll see:

- Original webcam with GREEN BOXES around moving objects
- Cleaned foreground mask
- Real-time tracking of movement

■ Try:

- Move around → box follows you
- Multiple people → multiple boxes
- Stay still → box disappears

Limitations of Traditional CV

Challenge 1: Manual Parameter Tuning

- Every environment is different:
 - Different lighting → different thresholds
 - Different objects → different parameters
 - Different cameras → different settings
- You could be spending hours tweaking parameters!

```
params.minThreshold = 10    # Why 10?  
params.maxThreshold = 200   # Why 200?  
params.minArea = 1000       # Why 1000?  
params.minCircularity = 0.5 # Why 0.5?
```

Challenge 2: Lighting Changes

- Morning vs. Evening:
 - Same object, different appearance
 - Parameters that worked at 9am fail at 5pm
 - Need constant re-tuning
- Shadow problems:
 - Shadows detected as objects
 - False positives everywhere
 - Difficult to filter out
- **Solution?** Machine learning adapts automatically!

Challenge 3: Complex Patterns

- Traditional CV struggles with:
 - **Cluttered backgrounds:** too much visual noise
 - **Partial occlusion:** object partially hidden
 - **Varying poses:** person standing vs. sitting
 - **Similar colors:** object blends with background
- Example:
 - Blob detection can find a ball on plain background
 - But fails to find ball in grass with same color
- **With ML**, the model learns complex patterns

Challenge 4: Limited Generalization

- **Problem:** Every object type needs new code
- Each requires custom parameters and logic vs. **ML approach:** One model detects many object types

```
# Detect red ball
detect_red_ball(frame, threshold_red_min, threshold_red_max)

# Detect blue ball
detect_blue_ball(frame, threshold_blue_min, threshold_blue_max)

# Detect hand
detect_hand(frame, skin_color_range, shape_params)

# Detect face
detect_face(frame, face_cascade, eye_cascade)
```

Challenge 5: Requires Expertise

- To use traditional CV well, you need to understand:
 - Color spaces (RGB, HSV, LAB)
 - Morphological operations
 - Thresholding techniques
 - Contour properties
 - Kernel design
 - Camera geometry and more
- ML can simplify this approach without hand-crafting and make the barrier lower to enter

When Traditional CV Works Well

- Controlled environments (lab, studio)
- Consistent lighting (indoor, fixed lights)
- Simple backgrounds (plain walls, solid colors)
- Known objects (specific markers, colors)
- Real-time requirements (very fast processing)
- Limited compute (embedded systems, Arduino)
- For example: QR code detection, line following robots, color-based sorting and motion detection in fixed camera

Machine Learning Overcomes These Limitations

■ Automatic Feature Learning

- No manual parameter tuning
- Learns what's important from data

■ Handles Complexity

- Works in cluttered scenes
- Robust to lighting changes
- Handles occlusion

■ Generalizes Better

- One model, many object types
- Learns from examples

■ Adapts to New Scenarios

- Retrain on new data
- Improves with more examples

See you next week!